Evidence, not verdicts: what 590 public PX4 crash logs can and cannot tell a drone fleet operator

Nagesh Chougule[a,*], Vibhav Ambardekar[a], Aditya Pathare[a], Atharv Waghmare[a], Shreyash Pawar[a], Piyush Bhamare[a]

[a]*Department of Mechanical Engineering, COEP Technological University, Pune, India*

Abstract

Flight logs are widely expected to show whether a drone crash was caused by the pilot or the aircraft. Across 590 uploader-rated PX4 crash logs from an index of 479,547 flights (590 rated → 520 on real hardware → 205 rule-evaluated → 148 modelled), we ask what evidentiary value logs carry for a fleet operator once evaluation guards against crash-window leakage and between-aircraft confounding. Confirmatory tests were declared in version control before analysis; analyses added during review are marked †. A fixed attribution rule reached a balanced accuracy of 0.47 on 112 held-out crashes (declared pass mark 0.70). †Cross-validated models gave AUCs of 0.41–0.50 (upper 95% bootstrap bounds ≤ 0.62); with 36 pilot-labelled crashes in the modelling sample, only AUCs above about 0.65 are statistically detectable at 80% power, and an operational verdict would need about 0.8. †On 43 expert-diagnosed logs the rule reached a balanced accuracy of 0.381 (exact 95% Clopper–Pearson bound 0.19–0.73; Bayesian 95% CrI 0.25–0.61). †Moving from cross-sectional to within-drone evaluation reduced AUC from 0.88 to 0.58 on the same flights, consistent with airframe and duration confounding. †Diagnostic checks were consistent with crash data leakage (exploratory, tuning fault sensitivity fell from 77.8% to 11.1%, $p = 0.031$, without the final 10 s; normal-flight false alarms fell from 13% to 5%). Drones that later crashed vibrated more than matched peers on older estimator metrics (mean rank 0.67, Holm $p = 0.016$); replication on newer accelerometer metrics was inconclusive ($p = 0.25$, 19 sets). †Screening at a pre-specified 10% flag rate gave positive predictive values of 2.26–2.47% (lift 3.3×, 40–44 inspections per crash) on a reference of 107 control drones. AeroFleet 2.0 presents longitudinal evidence without issuing automatic verdicts.

*Corresponding author.

Email address: nkc.mech@coeptech.ac.in (Nagesh Chougule)

Keywords: Unmanned aircraft systems, Flight log analysis, Crash investigation, Fault diagnosis, Data leakage, Declared protocols, Fleet assurance

1. Introduction

Small unmanned aircraft are now flown commercially for survey, inspection, agriculture and delivery, and the rules that govern them place weight on records. In the European Union and the United Kingdom an operator in the specific category must keep records of operations, maintenance and personnel for at least three years [1, 2]; India requires an accident to be reported within 48 hours [3], and the United States within ten days [4]. When something goes wrong, the onboard flight log is the richest record available. Open-source log viewers, commercial fleet platforms and, more recently, machine-learning and language-model prototypes promise to turn that log into an answer: what failed, and whether the pilot or the aircraft was responsible [5, 6].

The question matters beyond engineering curiosity. In aviation safety science, accidents are recognized as systemic, multi-causal events involving human factors, environmental conditions and organizational influences [7]. Commercial warranty claims, insurance settlements and regulatory filings nevertheless frequently compress these dynamics into a binary decision: whether the pilot or the aircraft was primarily responsible. An automatic verdict that looks objective can carry weight it has not earned. Fleet operators face a related but different question: not why one flight ended badly, but which of their drones are most likely to fail next and should be inspected first. A log-based method that is good at the second task need not be good at the first, and confusing the two can mislead both investigators and maintainers.

Whether logs can deliver on either expectation has, to our knowledge, not been tested at scale against real crashes with evaluation protocols that guard against the two known sources of optimism. First, a log that contains a crash also contains its consequences—saturated motors, extreme attitudes and estimator errors—so a detector can learn the crash itself rather than its cause, a form of crash-window data leakage [8, 9]. Second, comparing crash flights with other operators' good flights mixes the flight with the aircraft and the people using it, so a difference between groups need not say anything about the flight that crashed (between-aircraft confounding). Public fault datasets are dominated by simulation or injected faults [10, 11, 12], and accident statistics classify outcomes such as loss of control rather

than causes visible in a log [13, 14]; neither resource tests whether real crash logs, evaluated under leakage-free and within-aircraft protocols, can support the attribution and prioritization decisions that operators actually need.

The central question this paper asks of the largest public archive of real autopilot logs, PX4 Flight Review [15, 16], is: *what evidentiary value can flight logs carry for a drone fleet operator, once evaluation protocols guard against crash-window leakage and between-aircraft confounding?* We decompose this into four sub-tests:

- Q1 Do configuration settings differ between crash flights and normal flights, and can that difference indicate who was responsible?
- Q2 Can a log separate crashes caused by the pilot from crashes caused by the aircraft—and does this separation survive expert-diagnosis validation?
- Q3 Once within-aircraft and between-aircraft comparisons are distinguished, which comparison retains a usable signal of pre-crash deterioration?
- Q4 Do expert diagnoses agree with uploader labels, and do automatic diagnostic checks find the faults experts name without benefiting from the crash window?

Because language-model assistants are now proposed for exactly this task, we also report a controlled evaluation of small local models used as an attribution review panel, including a test for voting-consensus manipulation.

The contributions are as follows. (i) We introduce and apply a *crash-window ablation and within-aircraft Group-CV evaluation protocol*: we remove the final crash window from diagnostic checks (quantifying the sensitivity drop from 77.8% to 11.1%, $p = 0.031$) and replace cross-sectional comparisons with grouped cross-validation across the same aircraft's history (reducing apparent AUC from 0.88 to 0.58); this reusable protocol is committed to public version control prior to analysis [17] so that others can apply it to any drone-log diagnostic. (ii) We assemble, to our knowledge, a small first validation set of real-flight crash logs from public PX4 archives that pairs uploader responsibility labels with independent expert forum diagnoses ($\kappa \geq 0.73$, 148 threads, 43 downloadable logs). (iii) We show that with these rules, models and labels, cross-validated classification yielded AUCs between 0.408 and 0.501 (upper 95% bootstrap bounds 0.510–0.619); with 36 pilot-labelled crashes in the modelling sample, only AUCs above about 0.65 can be statistically detected at 80% power, far below the roughly 0.8 an operational attribution verdict requires. Agreement with expert forum diagnoses

did not exceed chance (balanced accuracy 0.381, exact 95% Clopper–Pearson bound 0.19–0.73; pilot recall 1 of 5). This constitutes a prospectively declared negative result, not a post-hoc reinterpretation. Furthermore, apparent diagnostic accuracy is consistent with crash-window leakage. (iv) We report one partial positive screening signal: drones that later crash are rougher than matched peers on older estimator metrics (mean rank 0.67, Holm $p = 0.016$); replication on newer accelerometer metrics was inconclusive ($p = 0.25$, 19 sets, severely underpowered). Stratum-matched fleet screening yields an operational PPV of 2.26–2.47% (lift 3.3×, 40.5–44.2 inspections per crash) at a 10% flag rate on a reference of 107 control drones—a maintenance prioritization signal, not an operational warning. (v) We show how these results translate into design implications for fleet assurance tools that present longitudinal evidence and rank drones without issuing automatic verdicts. Code, declarations, derived features and labels are released.

2. Related work

Fault datasets. Table 1 compares our data with the main public resources. ALFA provides 47 fixed-wing flights with injected engine and control-surface failures [10]. RflyMAD covers 5,629 multicopter fault cases with eleven fault types, of which 497 are real flights and the rest software- or hardware-in-the-loop simulations [11]. UAV-SEAD offers about 1,396 real PX4 logs annotated for state-estimation anomalies [12], used by later anomaly-mining work [18]; a recent testbed quantifies the discrepancies between simulated and real PX4 logs [19]. These resources target anomaly or fault detection under known faults. To our knowledge, none of these public resources labels real crashes by responsibility or separates the pre-crash window from the crash.

Fault detection and diagnosis. Surveys of fault detection for UAVs and robots cover model-based and data-driven methods and stress how hard robotic diagnosis is in the field [20, 21]. Causal models built from autopilot documentation have been used to find actual causes of failures in ArduPilot logs [22]. A study of 569 bugs in PX4 and ArduPilot found parameter handling among the root-cause types of drone-specific bugs [23]. Maintenance-oriented reliability analysis of UAVs models the passage from partial degradation to failure [24], the view our between-drone result supports.

Accident analysis. Analyses of civil drone occurrences find equipment problems and loss of control prominent [13]. Human-factors frameworks for re-

Table 1: Public UAV fault and anomaly data compared with this study.

Resource	Flights	Faults or labels	Real crashes labelled by cause
ALFA [10]	47 fixed-wing flights	Injected engine and control-surface failures	No
RflyMAD [11]	5,629 cases (497 real)	11 injected fault types	No
UAV-SEAD [12]	about 1,396 real PX4 logs	State-estimation anomalies (4 classes)	No
This study	479,547-log index; 590 crash-rated logs; 148 expert-diagnosed threads	Uploader responsibility labels; expert causes	Yes (uploader; partly expert-checked)

motely piloted aircraft (such as the Human Factors Analysis and Classification System, HFACS) model mishaps as multi-causal sequences involving organizational influences, environmental preconditions, and supervisory failures alongside direct operator actions [7, 25]. In operational practice, post-incident reporting and warranty claims often collapse these complex systemic interactions into a single nominal party: pilot versus aircraft. Because such classifications depend heavily on the rater and framing, we evaluate agreement against expert community diagnoses.

Language models for diagnosis. Language-model agents have been applied to simulation testing and log analytics for small UAVs [5], and learned models to classifying problems in drone log messages for forensics [6]. Multi-agent debate has been proposed to improve the factuality of model outputs [26], while indirect prompt injection shows that text inside data can steer model behaviour [27]. We test both properties on our task.

Methodology. Leakage, where information unavailable at prediction time enters training or evaluation, is a known source of irreproducible machine-learning results [8, 9]; label noise likewise biases evaluation [28]. In the drone-log domain we identify a domain-specific variant: *crash-window leakage*, in which the physical consequences of the crash (saturated actuators,

estimator divergence, attitude extremes) contaminate features used to evaluate pre-crash diagnostics. Matching reduces confounding in observational comparisons [29], and declaring hypotheses and analysis scripts in version control prior to running tests separates confirmatory from exploratory analysis [17]. We apply these ideas together as an evaluation protocol for drone logs.

Tools for operators. Retrieving similar past cases and reusing their solutions is the basis of case-based reasoning [30]; ranking text by word overlap with BM25 is a standard, deterministic retrieval method [31]. A hand-held miniature of the environment lets a person grasp and inspect a spatial scene at a glance [32], the idea behind the tabletop view in Section 6. We use these as engineering choices and test them only as far as stated.

3. Data

3.1. Archive and labels

PX4 Flight Review is a public service to which PX4 users upload onboard logs (ULog format) for plotting and sharing [16]. We downloaded its public index on 20 October 2026: 479,547 logs (32,793,172 bytes, SHA-256 `1bbb01c3…a92d6`), of which 442,929 came from real hardware rather than simulation, from 38,320 distinct vehicle identifiers. Uploaders may rate a log. A total of 590 logs carry an uploader crash rating (402 aircraft-caused, 188 pilot-caused); Table 2 details the 520 crash-rated logs from real hardware with duration $\geq 10\,\mathrm{s}$ and complete firmware version records. These uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments. A notable discrepancy appears between uploader ratings and expert conclusions: while uploaders attributed 31.9% of rated crashes to the pilot (188 of 590), forum experts blamed the pilot in only 6.1% of all threads (9 of 148) and 8.6% of decided threads (9 of 105, Section 5.6). Across all 148 real crash threads, download attempts yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Cross-tabulating these 43 logs against archive ratings reveals that 9 carried uploader aircraft crash ratings (`crash_sw_hw`, with 100% expert agreement: all 9 diagnosed as aircraft faults), 33 were unrated in the archive index (28 aircraft, 5 pilot), 1 was rated unsatisfactory (aircraft), and exactly 0 were uploaded with a `crash_pilot` rating. This pattern is consistent with,

but not statistical evidence of, reluctance to self-identify pilot fault: all 5 pilot logs were unrated in the index against 28 of 38 aircraft logs unrated (one-sided Fisher exact $p = 0.25$). Across the wider set of all 148 forum threads, 14 referenced logs that carried an uploader aircraft crash rating in the archive index (experts agreed with the aircraft fault in 11 and were unclear in 3; none blamed the pilot); 9 of these 14 threads were among the 43 with surviving, downloadable log files (all 9 with complete expert agreement), while 5 had expired external links. This gap may reflect selection bias in which incidents operators submit for community assistance, or a narrower definition of pilot error among autopilot developers compared to end users. Furthermore, forum diagnoses represent informed community opinion rather than independently verified accident inspection ground truth; original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft threads and 5 of 9 pilot threads. Logs rated "good" or "great" served as normal flights. Logs were parsed with pyulog. We store derived features and log identifiers, not the logs.

A ULog file contains the parameters in force when logging started, any parameter changes during logging, timestamped messages published by the autopilot's modules, and logged text messages with a severity level. The analyses below use the vehicle status (arming state, navigation mode, failsafe flags), attitude and body-rate topics, rate setpoints, inertial-sensor status, actuator outputs, battery status, GPS position and the land detector. The vehicle identifier in the index is derived from the flight controller's hardware serial number; it identifies a flight controller, which in practice usually means one airframe, though a controller can be moved between frames.

Crash-rated logs differ from well-rated logs in composition (Table 2): they are shorter (median 54 s against 388 s, partly because a crash ends the log) and spread over more firmware generations. All between-group comparisons below therefore use either matching or within-drone designs, and we do not pool across firmware generations where measurements differ.

3.2. Samples

Figure 1 and Table 3 show how each analysis sample was drawn. We evaluate two primary classification tasks: Task A (separating crash flights from normal flights) and Task B (binary attribution of crash responsibility between drone and pilot). We distinguish two distinct cohorts that each happen to number 148: (1) the leak-curve and attribution modeling sample of 148 crashes and 139 normal flights at $m = 0$ s (Section 4), and (2) the

Table 2: Composition of crash-rated and well-rated logs from real hardware in the archive.

	Crash-rated	Rated good or great
Logs	520	21,112
Median duration	54 s	388 s
Most frequent years	2018 (91), 2025 (89), 2019 (87), 2017 (57)	2018 (10,571), 2017 (5,748), 2019 (2,910)
Most frequent PX4 versions	1.8 (70), 1.9 (66), 1.15 (65), 1.11 (65)	1.7 (6,299), 1.8 (5,785), 1.6 (4,265)
Vehicle types	quadrotor 320, fixed wing 71, hexarotor 38	quadrotor 19,890, fixed wing 620, hexarotor 291

expert community sample of 148 crash discussion threads on the PX4 forum (Section 5.6). The attribution rule was evaluated on 205 crashes (93 development and 112 held-out). All 148 crashes entering the statistical attribution models are a strict complete-case subset of these 205 crashes (100% overlap); 57 crashes were excluded from modeling because required ULog topics were unrecorded or the flight window before cut-off was shorter than 10 s. In the leak-curve and attribution modeling sample, sample sizes across cut-off margins $m \in \{0, 2, 5, 10, 20, 30\}$ s reflect progressive attrition as flights with duration under 10 s before cut-off are dropped: for Task A (crash vs. normal), 148 vs. 139 ($m = 0$ s), 142 vs. 137 ($m = 2$ s), 135 vs. 133 ($m = 5$ s), 114 vs. 130 ($m = 10$ s), 100 vs. 128 ($m = 20$ s), and 89 vs. 124 ($m = 30$ s); for Task B (drone vs. pilot responsibility among crashes), 112 vs. 36 ($m = 0$ s, $n = 148$), 106 vs. 36 ($m = 2$ s, $n = 142$), 101 vs. 34 ($m = 5$ s, $n = 135$), 87 vs. 27 ($m = 10$ s, $n = 114$), 73 vs. 27 ($m = 20$ s, $n = 100$), and 63 vs. 26 ($m = 30$ s, $n = 89$). A constant complete-case subset of 213 flights (89 crashes and 124 normal flights; 63 drone and 26 pilot crashes) with pre-crash duration ≥ 40 s was evaluated across all six margins without attrition.

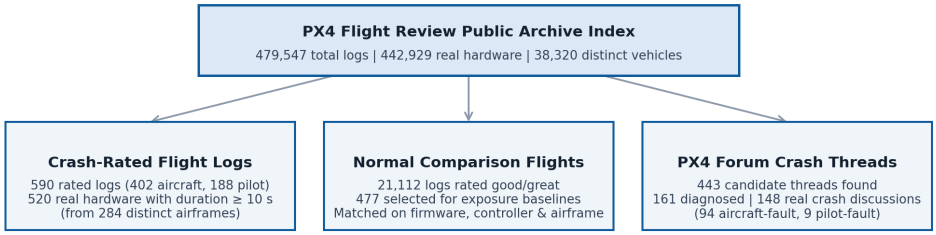
Initial cross-sectional samples for Q1 and Q2 selected flights with PX4 1.10 or newer and at least 5 s of flight; normal flights were matched to crashes on vehicle type and drawn from vehicles without any crash rating. For the longitudinal within-drone and between-drone tests (Q3b, Q3c), only the eligible case count ($n = 12$), not any outcome or feature data, was inspected before deciding to lift the firmware restriction to avoid sample starvation; flights were instead strictly matched within each drone's firmware generation and measurement source. For the within-drone test every crash with a vehi-

cle identifier and at least three earlier non-crash flights of 30 s or more was eligible (101 crashes; up to the five most recent earlier flights each). For the between-drone test each crashed drone was matched with up to two drones without any crash-rated log that shared the flight-controller hardware, the PX4 major.minor version and the vehicle type (66 matched sets, 124 control slots drawn from 107 unique non-crash control drones, three randomly chosen flights each, seed 7). Control drones were sampled without replacement within each matched set. The three exposure flights of a crashed drone were its most recent non-crash flights dated on or before the crash; flights on the crash day were included in the declared test because the archive does not give their order within a day, and removed in a sensitivity analysis.

Not all selected flights enter every test. In the within-drone candidate pool (101 crashes with earlier flights), 34 crashed drones were dropped because they lacked multiple valid flights or had duration under 10 s before cut-off, leaving 67 multi-flight drones (372 total flights: 67 crash and 305 earlier flights). Among these 372 flights, 55 individual flight logs could not be read or had under 10 s before cut-off; requiring that crash and earlier flights share the same measurement source left 61 analysed drones for vibration, and restricting to rotary-wing vehicles with rate setpoints left 46 analysed drones for rate tracking. In the between-drone sample (66 matched sets), 71 flights were unreadable or too short; a drone's value requires at least two readable flights, and case and controls must share the measurement source, leaving 60 analysed sets for vibration and 47 for tracking. In the between-drone sample, 124 matched control slots contributed 372 flights (124×3), distinct from the 372 flights (67 crash flights and 305 earlier flights) evaluated in the within-drone longitudinal model.

3.3. Expert diagnoses

We searched the PX4 discussion forum for threads that link a Flight Review log and mention a crash (443 threads, 410 with a log link). For the 161 threads with a reply from PX4 staff, the development team or a member at the forum's highest trust levels (148 of them describing a real crash), we recorded the expert's conclusion (aircraft, pilot, both, unclear, no diagnosis), the cause named, who gave it, whether the original poster confirmed it, and the sentence stating the diagnosis. Recording was AI-assisted and reviewed by an author; 123 of 124 recorded sentences were found verbatim in their thread by script. A second author then coded all threads blind to the first recording, with the same categories; a script confirmed that 150 of 151 coded



Analysis Stream (Protocol)	Initial Candidate Pool	Exclusions & Attrition Criteria	Analyzed Sample Size
Q1: Parameter Settings	100 crash, 477 normal	43% crash logs lack factory default records	57 crash, 113 normal
Q2: Fixed Rule Attribution	205 crashes, 143 normal	None (evaluates all available crashes: 93 dev	205 crashes, 143 normal
Q2/Q3a: Data Leakage Curve	205 crashes, 143 normal	57 dropped (missing topics or duration < 10	148 crash, 139 normal (m=0 s) 89 crash, 124 normal (m=30 s)
Q3b: Within-Drone Longitudi	101 crashed drones	34 dropped (single flight recorded or duratior	67 multi-flight drones (372 flights) 61 vibration sets, 46 tracking sets
Q3c: Between-Drone Matched	66 matched sets (101 case	6 sets dropped (unmatched sensor metric ac	60 vibration sets (124 controls, 372 f 47 rate-tracking sets
Q4: Expert Community Cases	148 real crash threads	105 logs unhosted or expired download links	43 downloadable logs (38 aircraft, 5 p 37 aircraft logs with fault categories

Figure 1: From the public index to each analysis. Counts are logs unless stated.

rows quoted their own thread or recorded no diagnosis. On the 147 threads both coded, agreement (Cohen's kappa, 95% bootstrap interval) was 0.81 (0.71–0.88) for the expert's conclusion, 0.73 (0.65–0.81) for the cause and 0.79 (0.69–0.88) for whether the poster confirmed it, above the declared pass mark of 0.60. The two coders settled the 45 threads that differed on conclusion or cause by discussion, which changed 13 labels; counts and tests below use the settled labels, and the first recording led to the same conclusions (both are in the repository). The searches combined the Flight Review host names with the words crash, crashed, flyaway, fell, flipped and lost control, and every returned thread was retrieved through the forum's public interface. A reply counted as expert when its author was marked as staff, belonged to the development team group or held one of the forum's two highest trust levels. Only public posts were used; usernames and post text are not redistributed.

4. Methods

4.1. Declared test protocols

For each test, the selection, features, statistic, threshold and pass mark were committed and pushed to a public git repository before the data it used were downloaded or analysed. Unlike third-party timestamped registries (such as the Open Science Framework), commit timestamps in self-hosted git remain author-controlled until an independent snapshot is indexed; the git commit history establishes an immutable cryptographic SHA-1 chain in version control, and archived snapshots have been deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`) alongside the repository's git commit hashes, and will be minted into a Zenodo archive DOI upon publication. One amendment was data-dependent: after observing that the firmware filter of the within-drone test left only 12 eligible cases under the initial ≥ 1.10 filter (without inspecting any outcome data), the firmware filter was removed and a fallback for older firmware was added. One coding error was corrected after a first run (logs whose CPU-load record lacked the load field were being skipped); no threshold changed. Secondary sensitivity analyses and diagnostic evaluations added during peer review are explicitly designated as exploratory and documented with commit hashes in Section 5.8.

4.2. Crash moment and analysis window

For rotary-wing aircraft the crash moment t_c is the first time after take-off at which the tilt $\theta = \arccos\left(1 - 2(q_x^2 + q_y^2)\right)$, computed from the attitude quaternion, exceeds 60°; for fixed-wing aircraft, and when no such tilt occurs, it is the end of the armed period. Normal flights use the end of the armed period. Take-off is the first sample after arming at which the land detector reports the vehicle airborne. An analysis with cut-off m uses the window from take-off to $t_c - m$ and is skipped if the window is shorter than 10 s. Because normal flights include the complete descent, landing and disarm sequence while crash windows terminate at impact or tilt, this introduces an inherent windowing asymmetry that contributes to differences in duration and whole-window aggregates.

4.3. Features

Table 4 defines the features. Vibration is not measured the same way across firmware generations: newer logs report an accelerometer vibration metric, older ones an estimator vibration estimate. Comparisons are therefore only made between flights measured with the same source, and the source is recorded with every value. The rate-tracking error falls back to the attitude message's body rates when the angular-velocity topic is absent.

4.4. Tests

Q1 settings. For logs that carry the firmware's record of default values, a protection was counted as disabled when its value differed from the default and switched it off (for example a failsafe action of "none"). Crash and normal logs were compared with Fisher's exact test.

Q2 attribution. Task B is defined as the binary attribution of crashes between pilot and aircraft fault (contrasted with Task A, which evaluates separating crash flights from normal flights). The declared rule labelled a crash "aircraft" if any of eight signs fired before t_c (a critical battery warning or remaining charge below 10%; any motor at 95% or more for at least 1 s in the last 10 s; more than 50 clipping events; poor GPS in a GPS-dependent mode; a failure-detector flag; an error-level message; CPU load above 95%; an active failsafe), otherwise "pilot" if the mode at t_c was pilot-flown, otherwise "unknown". The pass mark was a balanced accuracy of at least 0.70 on decided cases, answering at least 60% of crashes and being wrong on at most 20%. A model-based check used L2-regularised logistic regression on all features ($\lambda = 1$, standardised inputs, median imputation within training folds), evaluated by stratified five-fold cross-validation repeated five times and summarised by the ROC AUC [33]. As an exploratory sensitivity analysis, non-linear feature interactions were evaluated using depth-3 gradient-boosted decision trees.

The signs were chosen before the analysis to cover common failure modes of PX4 multirotors: power, propulsion, vibration, navigation, software and failsafes. The pilot rule was deliberately simple: if no aircraft sign fired and the vehicle was in a mode in which the pilot controls attitude or position directly, the crash was attributed to the pilot. The rule was fixed before it was run on any crash log; only the list of message types available in the logs had been inspected.

Q3a crash window. The same model separated crash from normal flights at cut-offs $m \in \{0, 2, 5, 10, 20, 30\}$ s. We declared that the AUC would fall by at least 0.10 between $m = 0$ and $m = 10$ s if crash consequences drove it.

Q3b and Q3c rank tests. For a crashed drone i with value x_{i0} and comparison values $x_{i1}, \ldots, x_{in_i}$, the scaled rank is

$$r_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1}\left[x_{ij} < x_{i0}\right], \tag{1}$$

which is 0 when the crashed drone has the lowest value and 1 when it has the highest; ties count against the hypothesis. A rank statistic was chosen for three reasons: the two vibration sources have different units and scales, so only orderings are comparable across drones; the number of comparisons differs between drones, which the exact null handles directly; and the distributions are skewed, with a few very rough flights, which would dominate a mean difference. In the within-drone test (Q3b) x_{i0} is the crash flight, cut 10 s before t_c, and the comparisons are the same drone's earlier flights, each cut 10 s before the end of its armed period. In the between-drone test (Q3c) x_{i0} is the median over the crashed drone's three most recent pre-crash flights and the comparisons are the medians of the matched controls. Under no difference each r_i is uniform on $\{0, 1/n_i, \ldots, 1\}$ with mean 0.5, and $\bar{r}$ estimates the probability that the crashed drone exceeds a comparison, the quantity estimated by the Mann–Whitney statistic [34]. One-sided p-values come from 100,000 draws of this exact null; 95% confidence intervals from 10,000 bootstrap resamples of drones [35]. For the proportion of sets in which the crashed drone is the roughest, the expected null rate depends on the mix of set sizes: among the 60 analysed vibration sets, 35 had two matched controls (null probability 1/3) and 25 had one control (null probability 1/2), giving an exact Poisson-binomial expected null proportion of $(35 \times \frac{1}{3} + 25 \times \frac{1}{2})/60 = 40.28\%$ (null mean 24.167, SD 3.745). For duration adjustment in sensitivity analysis, log(vibration) was regressed on log(flight duration) within each measurement source across all exposure flights, and the rank test was repeated on the residuals. Each test had two primary features (vibration, rate-tracking error) at $\alpha = 0.025$.

Q4 expert diagnoses. Expert conclusions were compared with uploader labels. Fixed deterministic checks for the five physical failure modes named by experts were evaluated: (i) *Tuning*: combined roll and pitch rate-tracking

RMS error $> 0.30\,\mathrm{rad/s}$ over at least 50 samples; (ii) *Vibration*: total accelerometer clipping counts > 50 or 95th-percentile `accel_vibration_metric` $> 30\,\mathrm{m/s^2}$; (iii) *GPS/Compass*: `mag_test_ratio` > 1.0 in `estimator_status` for $\geq 1.0\,\mathrm{s}$ cumulative, or horizontal position error eph $> 5\,\mathrm{m}$ or satellites < 6 for $\geq 5.0\,\mathrm{s}$ cumulative while in a GPS-dependent mode; (iv) *Motor/ESC*: any motor output at $\geq 95\%$ authority (or PWM $\geq 1950\,\mu\mathrm{s}$) for $\geq 1.0\,\mathrm{s}$ cumulative; (v) *Battery*: battery warning level ≥ 2 (critical/emergency) or state of charge $< 10\%$. Checks were declared useful if they fired on at least 50% of logs with that diagnosis and at most 10% of normal flights, evaluated first on the full window ($m = 0\,\mathrm{s}$) and then with the final $10\,\mathrm{s}$ removed ($m = 10\,\mathrm{s}$).

Language-model panel. The panel tested a design proposed for advisory diagnosis: several reviewers, each restricted to one slice of evidence (navigation, pilot inputs, GPS, battery, settings), vote on a fixed list of six causes and must cite numbered facts extracted from the log. Robustness was evaluated by replacing one reviewer with an adversary arguing for a chosen wrong cause, and planting hostile strings in uploader-controlled fields. Full details are provided in Section 5.7.

Multiplicity. Seven confirmatory tests produce p-values. Holm's procedure [36] is reported over all seven in addition to each test's declared α.

5. Results

5.1. Configuration settings (Q1)

Of 57 crash logs and 113 normal logs carrying a firmware-default record, 29 (51%) and 52 (46%) had at least one protection disabled relative to the default (Fisher $p = 0.63$). Pilot-caused crashes had a protection disabled more often than aircraft-caused ones: 14 of 20 (70%) against 15 of 37 (41%), an observed difference of +29.5% (95% CI 3.9–55.0%, odds ratio 3.42, Fisher's exact two-sided $p = 0.052$). This contrast is exploratory, not part of the pre-declared confirmatory family, and does not reach statistical significance unadjusted (Fisher's exact two-sided $p = 0.052$). Furthermore, default parameter records were present only on newer firmware (missing for 43% of the 100 crash logs); on the 57 available cases, treating disabled protections as a pilot-attribution classifier yields a sensitivity of 70.0%, a specificity of 59.5%, and a balanced accuracy of 0.647, below the declared 0.70 pass mark. None of 100 crash logs changed a safety-critical or flight-limit setting during the flight.

5.2. Attributing a crash to the pilot or the aircraft (Q2)

The fixed rule reached a balanced accuracy of 0.57 on 93 development crashes (35% wrong) and 0.47 on 112 held-out crashes (47% wrong), against the declared 0.70, and labelled 39% of 143 normal flights as aircraft faults (Table 5). The signs fired at similar rates whichever party the uploader blamed (Fig. 2): motor saturation in 23% of aircraft-caused and 29% of pilot-caused crashes (1% of normal flights), battery warnings in 15% and 18% (3%). Error messages and failsafes were no more frequent in crashes than in normal flights. Two signals noticed after the development run did not replicate on held-out crashes: accelerometer clipping (24% against 8%, $p = 0.045$) and manual mode at the crash (18% against 10%, $p = 0.165$).

In Task B, sample sizes decline across margins as flights under 10 s are excluded: 112 drone and 36 pilot crashes at $m = 0$ s ($n = 148$), 106 vs. 36 at $m = 2$ s ($n = 142$), 101 vs. 34 at $m = 5$ s ($n = 135$), 87 vs. 27 at $m = 10$ s ($n = 114$), 73 vs. 27 at $m = 20$ s ($n = 100$), and 63 vs. 26 at $m = 30$ s ($n = 89$). On the full sample ($n = 148$ at $m = 0$ s, 75.7% drone-blamed), pooled cross-validated logistic regression yielded an ROC AUC of 0.408 (bootstrap SE 0.062, 95% CI 0.288–0.530, upper 95% bound 0.510) at $m = 0$ s, 0.422 (SE 0.072, 95% CI 0.283–0.566, upper bound 0.542) at $m = 10$ s ($n = 114$), and 0.501 (SE 0.073, 95% CI 0.358–0.645, upper bound 0.619) at $m = 30$ s ($n = 89$). Evaluated on the constant complete-case sample of 89 crashes (63 drone, 26 pilot; 70.8% drone-blamed) plotted in Fig. 3, pooled cross-validated logistic regression yielded an AUC of 0.466 (bootstrap SE 0.072, 95% CI 0.325–0.607, upper bound 0.584) at $m = 0$ s, 0.489 (SE 0.074, 95% CI 0.343–0.633, upper bound 0.611) at $m = 10$ s, and 0.501 (SE 0.073, 95% CI 0.358–0.645, upper bound 0.619) at $m = 30$ s.

To calibrate whether these below-chance AUCs represent an inverted physical signal or finite-sample noise, we computed a 1,000-iteration permutation null distribution on Task B by randomly shuffling crash attribution labels: the null distribution had mean 0.497 (SD 0.090) with an empirical 95% interval of 0.313–0.660. The permutation null used the same pooled statistic (mean 0.497), so the below-chance values are not a bias of the statistic; rather, with 36 pilot-labelled crashes against 112 drone crashes in the modelling sample, below-chance point estimates reflect finite-sample variance under class imbalance rather than an inverted physical relationship. Evaluating depth-1 decision stumps on Task B yielded similar results: AUC of 0.430 at $m = 0$ s (bootstrap SE 0.062, 95% CI 0.310–0.551, upper bound

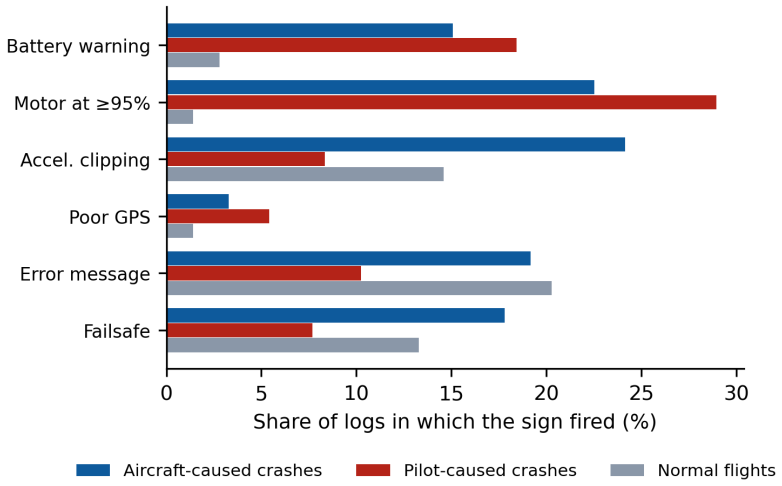


Figure 2: How often each aircraft-fault sign fired in held-out crashes, by uploader label, and in normal flights.

0.534), and 0.413 at $m = 10\,\mathrm{s}$ (SE 0.069, 95% CI 0.281–0.548, upper bound 0.528).

With 36 pilot-labelled crashes in the modelling sample, AUCs below roughly 0.65 cannot be distinguished from chance (permutation 97.5th percentile 0.660). The observed bootstrap upper bounds were 0.510–0.619. An operational attribution verdict would need an AUC of about 0.8 or higher, which these models do not approach. We therefore report that these rules, models and labels give no usable attribution signal, not that attribution is impossible.

5.3. The crash window (Q3a)

In the primary declared sample, crash flights were separated from normal flights with a cross-validated AUC of 0.88 (SD 0.01) at $m = 0\,\mathrm{s}$ ($n = 148$ crashes, 139 normal flights), 0.88 at $m = 2\,\mathrm{s}$ (142 vs. 137), 0.88 at $m = 5\,\mathrm{s}$ (135 vs. 133), 0.87 (SD 0.01) at $m = 10\,\mathrm{s}$ (114 vs. 130), 0.86 at $m = 20\,\mathrm{s}$ (100 vs. 128), declining to 0.86 at $m = 30\,\mathrm{s}$ (89 vs. 124); the declared drop of 0.10 was not observed (observed drop of 0.01). To evaluate whether this apparent stability was an artefact of sample attrition as shorter flights dropped out at larger margins, we evaluated the cross-validation on the fixed constant subset of 213 flights with duration $\geq 40\,\mathrm{s}$ (89 crashes, 124 normal flights; plotted in Fig. 3) across all six margins. On this constant sample, model AUC

declined smoothly from 0.895 (SD 0.007) at $m = 0\,\mathrm{s}$ to 0.876 at $m = 2\,\mathrm{s}$, 0.873 at $m = 5\,\mathrm{s}$, 0.868 at $m = 10\,\mathrm{s}$ (a drop of 0.027), 0.862 at $m = 20\,\mathrm{s}$, and 0.864 at $m = 30\,\mathrm{s}$. The absence of a large drop in Task A AUC (0.895 to 0.868) indicates that the cross-sectional model does not rely primarily on transient crash-impact telemetry. Instead, classification is likely influenced by flight envelope differences and airframe characteristics that correlate with duration (Table 7). Although flight duration was deliberately excluded from the 11 normalized model features, duration alone separates crash from normal flights with an AUC of 0.662 at $m = 0\,\mathrm{s}$ (crash median 71.0 s vs. normal median 169.0 s) and 0.634 at $m = 10\,\mathrm{s}$, and correlates significantly with several normalized features: longer flights exhibit larger 95th-percentile tilt ($\rho = +0.470$ at $m = 0\,\mathrm{s}$, $+0.484$ at $m = 10\,\mathrm{s}$), greater voltage sag ($\rho = +0.321$ and $+0.357$), and larger stick deflections ($\rho = +0.363$ and $+0.233$), while shorter flights show higher rate-tracking errors ($\rho = -0.248$ at $m = 0\,\mathrm{s}$). The absence of a large collapse holds on identical flights, confirming that changing sample composition across cut-offs did not mask a collapse.

Leakage was nevertheless present for individual physical checks. The rate-tracking feature alone fell from an AUC of 0.81 to 0.69 as the crash was excluded, while vibration stayed between 0.72 and 0.77 (Table 7). A fixed tuning check that found 10 of 12 candidate expert-diagnosed tuning faults (7 of 9 on paired surviving logs) with the crash included found only 1 of 9 once the last 10 s were removed (Section 5.6).

Crucially, the high cross-sectional model separation (AUC 0.88) does not demonstrate unconfounded anticipation of an impending crash: crash and normal flights differ fundamentally in duration (median 54 s against 388 s in the broader archive; 71 s vs. 169 s in the complete-case sample), firmware era distribution, and operational flight envelope. Table 7 gives the single-feature AUCs. Values below 0.5 indicate features that are lower in crash flights: crash flights reached lower tilt and smaller stick deflections over the whole flight than normal flights, consistent with short, aborted or cautious testing flights. Table 6 reports a step-by-step decomposition isolating what drives the cross-sectional separation. Grouping cross-validation folds by vehicle ID (Step 1) on the cross-sectional sample yields virtually identical separation (AUC 0.884), confirming that disjoint vehicle assignment across folds does not remove cross-sectional confounding when crash and normal flights originate from different operators. Removing the crash window (Steps 2 and 3) similarly retains high separation (AUC 0.880 and 0.863), while removing the tilt and stick boundary features (Step 4) reduces AUC modestly to

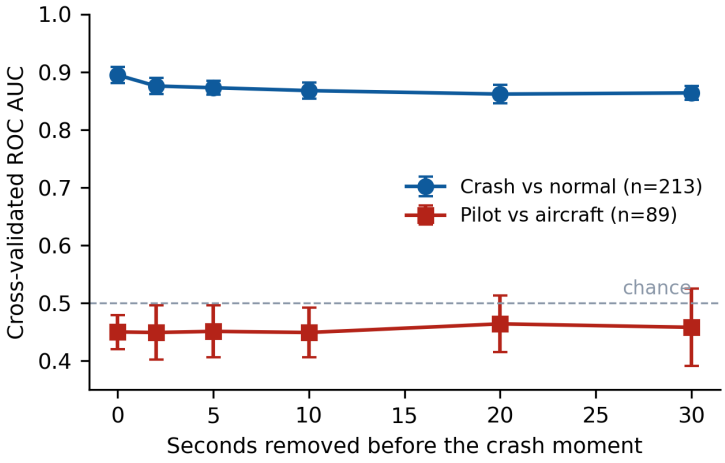


Figure 3: Cross-validated AUC on the constant subset ($n = 213$ for crash vs. normal, $n = 89$ for pilot vs. aircraft) as the end of the analysis window moves away from the crash moment. Bars: 95% bootstrap confidence intervals.

0.831. In contrast, when the exact same 11-feature model was evaluated *within the same drone* (Step 5)—separating each crashed drone's pre-crash window (cut 10 s before impact) from its own earlier flights across 67 multi-flight drones (372 flights: 67 pre-crash flights and 305 earlier flights)—the pooled Group-CV AUC fell to 0.580 (95% bootstrap CI 0.528–0.633), and the mean within-drone pairwise AUC was 0.605 (95% bootstrap CI 0.516–0.692). Matching on the airframe substantially attenuates the cross-sectional separation, consistent with airframe, duration and window-design differences rather than impending-crash precursors (operator identity was not modelled), in agreement with the within-drone rank test ($\bar{r} = 0.48$).

5.4. Within the same drone and between drones (Q3b, Q3c)

Relative to the same drone's earlier flights, the crash flight was neither more vibrating ($\bar{r} = 0.48$, 95% CI 0.38–0.58, $p = 0.67$, 61 drones) nor worse at tracking its commands ($\bar{r} = 0.50$, 0.38–0.63, $p = 0.47$, 46 drones; Fig. 4). Presented as an exclusion bound, the upper 95% confidence bound of 0.58 indicates absence of substantial pre-crash vibration surges within the same drone of the magnitude observed between drones ($\bar{r} = 0.67$). Relative to matched drones without a recorded crash, drones that later crashed vibrated more on their ordinary pre-crash flights ($\bar{r} = 0.67$, 0.57–0.77, $p = 0.0023$,

60 sets; Holm-adjusted $p = 0.016$), while the tracking error did not differ significantly ($\bar{r} = 0.57$, 0.44–0.70, $p = 0.15$, 47 sets). The vibration difference was clearest for aircraft-caused crashes ($\bar{r} = 0.67$, $p = 0.005$, 46 sets) and similar in size but not significant for pilot-caused ones ($\bar{r} = 0.64$, $p = 0.16$, 14 sets).

As a screening signal, the crashed drone was the roughest of its matched set in 33 of 60 sets (55.0%, Wilson 95% CI 42.5–66.9%, against 40.3% expected under the exact Poisson-binomial null; exact one-sided Poisson-binomial $p = 0.0137$). Furthermore, in 38 of 60 sets (63.3%, Wilson 95% CI 50.7–74.4%), the crashed drone exceeded the third quartile (Q3) of its matched controls. Sensitivity differed between measurement sources: 32 of 48 crashed drones (66.7%, Wilson 95% CI 52.5–78.3%) exceeded control Q3 on the older estimator metric, compared to 6 of 12 (50.0%, Wilson 95% CI 25.4–74.6%) on the newer accelerometer metric.

In practical fleet operations, positive predictive value (PPV) depends heavily on the operational flag rate and the fleet base rate. In the archive, 284 distinct vehicles accounted for all 520 real-hardware crash-rated logs out of 38,320 distinct vehicle identifiers, establishing an empirical vehicle crash base rate of 0.741% (284/38,320). When restricted to screenable multi-flight vehicles (airframes with ≥ 2 recorded flights, the operational prerequisite for pre-crash longitudinal screening), the crash base rate is 0.689% (182 crashed vehicles out of 26,421 multi-flight airframes). We emphasize that these base rates represent the lifetime probability of an airframe ever receiving a crash rating in the archive index, rather than an operational flight-hour incident rate, because the public archive does not record flight hours per airframe. Reported confidence intervals for PPV reflect sampling uncertainty in sensitivity (derived via 10,000 bootstrap resamples over crashed cases); the operational base rate is treated as a fixed fleet parameter. We evaluate screening performance under two distinct operational paradigms:

Empirical matched-set screening (45.1% flag rate): In our matched sets, flagging drones that exceed the upper quartile of their matched controls yields 63.3% sensitivity (38 of 60). Because sets contain 1 or 2 controls, the weighted null flag rate is 45.1%. At the 0.741% vehicle base rate, this yields an operational PPV of 1.04% (95% bootstrap CI 0.83–1.25%), an unflagged risk of 0.496%, a relative risk of 2.09× over unflagged airframes, a lift of 1.40× over base rate, and 96.4 inspections per detected crash (PPV 0.96%, 104 inspections per crash at the 0.689% screenable base rate).

Fleet-wide reference screening: Rather than comparing solely within small

local sets, an operator can screen airframes against an established reference distribution of non-crash control drones sharing the same flight controller hardware, firmware generation, and measurement source (20 strata, 107 unique control drones; overall AUC / mean percentile 0.689, 95% CI 0.619–0.758). Each airframe is ranked against control flights within its matching hardware and sensor stratum. At a 25% flag rate, this stratum-matched screening identifies 50.0% of crashed airframes (30 of 60, 95% CI 36.7–63.3%), yielding an operational PPV of 1.48% (95% CI 1.09–1.88%), an unflagged risk of 0.493%, a relative risk of 3.00×, a lift of 2.00× over the 0.741% base rate, and 67.5 inspections per detected crash (at the 0.689% screenable base rate: PPV 1.37%, 95% CI 1.01–1.74%, lift 1.99×, 73.1 inspections per crash). Tightening the threshold to the top decile (10% flag rate) achieves 33.3% sensitivity (20 of 60, 95% CI 21.7–45.0%), an operational PPV of 2.47% (95% CI 1.61–3.33%), an unflagged risk of 0.549%, a relative risk of 4.50×, a lift of 3.33×, and 40.5 inspections per detected crash (at the 0.689% screenable base rate: PPV 2.26%, 95% CI 1.48–3.06%, lift 3.28×, 44.2 inspections per crash). Under a broader source-matched reference (matching on measurement source alone; AUC 0.655, 95% CI 0.591–0.716), a 25% flag rate yields 40.0% sensitivity, PPV 1.19% (lift 1.60×, 84.3 inspections per crash), rising to 25.0% sensitivity, PPV 1.85% (lift 2.50×, 54.0 inspections per crash) at the 10% flag rate. In all scenarios, vibration screening functions as a broad maintenance inspection filter rather than an operational crash warning.

5.5. Robustness of the between-drone result (exploratory)

The vibration result held when controls were restricted to the same airframe as the case ($\bar{r} = 0.69$, $p = 0.008$, 39 sets), when case flights from the crash day were removed ($\bar{r} = 0.76$, $p = 0.003$, 25 sets) and after adjusting for flight duration ($\bar{r} = 0.69$, $p = 0.0005$, 57 sets; Fig. 5). Duration was barely correlated with vibration (Spearman $\rho = 0.05$ and -0.04 for the two measurement sources), although case flights were longer than control flights (median 289 s against 173 s). The sharp sample shrinkage when removing crash-day flights (from 60 to 25 sets) indicates that many pre-crash flights in the public archive occurred during the same flight session or day. The archive index did not record intra-day flight sequence, though parsing raw GPS timestamps in future work could establish whether crash-day flights immediately preceded the crash. This points to potential confounding by development or tuning sessions rather than progressive operational degradation: the "maintenance view" (vibration reflecting mechanical wear over

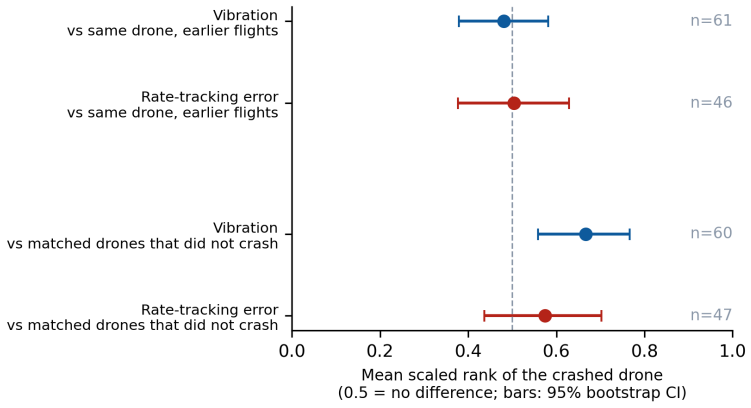


Figure 4: Mean scaled rank of the crashed drone against its own earlier flights (top) and against matched drones without a recorded crash (bottom). 0.5 indicates no difference.

time) competes directly with an "experimental airframe" view (poorly tuned or prototype drones vibrate more and crash during testing). Furthermore, matching was limited to flight controller hardware, PX4 major.minor version and vehicle type; it did not control for propellers, payload, airframe size, or operator identity, so controls from the same operator were not clustered.

The vibration difference was carried by the older estimator metric ($\bar{r} = 0.68$, $p = 0.005$, 48 sets); within the original cohort, the exploratory subset with the newer accelerometer metric showed a smaller, non-significant difference ($\bar{r} = 0.62$, $p = 0.191$, 12 sets; Fig. 5). A declared replication protocol on newer accelerometer logs selected 26 sets, yielding 19 usable matched sets (9 sets pooled from the initial cohort and 10 new cases entirely outside the original test). On the 10 new cases, the mean scaled rank was $\bar{r} = 0.50$ (95% CI 0.20–0.75, $p = 0.5760$), which is inconclusive given the wide confidence interval. Pooling all 19 usable sets gives $\bar{r} = 0.58$ (95% CI 0.39–0.76, one-sided $p = 0.251$). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if the true effect is $\bar{r} = 0.58$). Statistical sample sizing at one-sided $\alpha = 0.025$ indicates that detecting $\bar{r} = 0.67$ ($\Delta = 0.17$) with 80% power requires 55 matched sets; however, this assumes the discovery effect size was not inflated by winner's curse. If the true effect size is closer to the pooled estimate of $\bar{r} = 0.58$ ($\Delta = 0.08$), 247 sets are required. Because 247 matched sets exceed the capacity of the current single archive, resolving this effect definitively requires prospective multi-fleet

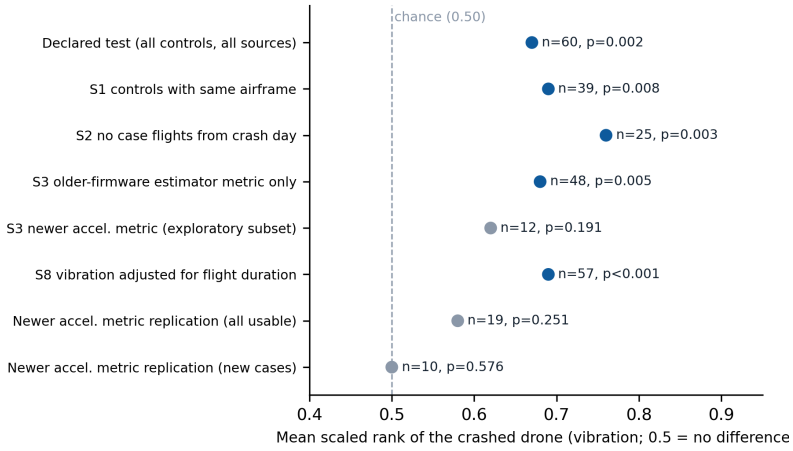


Figure 5: Between-drone vibration result under alternative specifications (exploratory). Specification labels: S1: airframe-matched controls; S2: excluding case flights from the crash day; S3: stratification by vibration measurement source (older-firmware estimator metric vs. newer accelerometer metric); S8: log-linear adjustment for flight duration. Blue: $p < 0.025$.

partner deployments.

5.6. Expert diagnoses (Q4)

Of 148 forum threads describing a real crash, the expert attributed it to the aircraft in 94 (64%), to the pilot in 9 (6%), to both in 2, and was unclear or gave no diagnosis in 43 (29%). Tuning or configuration was the cause named most often (26 of 94; 14 of the 47 confirmed by the poster), followed by GPS, compass or estimator problems (18) and motors, ESCs or propellers (16) (Fig. 6). Of 14 threads whose log carried an aircraft crash rating in the index, the expert agreed in 11 and was unclear in 3; none attributed the crash to the pilot. Download attempts across all 148 threads yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Of the 38 aircraft logs, exactly 35 were associated with checkable fault categories in Table 8 (12 tuning, 5 vibration, 5 GPS, 9 motor, 4 battery).

Table 8 compares the fault checks with the full flight window ($m = 0\,\mathrm{s}$) and with the last 10 s before the crash removed ($m = 10\,\mathrm{s}$). With the window ending 10 s before the crash, no check met the declared criterion. The tuning check illustrates the diagnostic leakage described in Section 5.3: across all

candidate logs with the full window, the check fired on 10 of 12 (83.3%). Across the 9 constant tuning logs with duration $\geq 10\,\mathrm{s}$ surviving the 10 s cut-off without sample attrition, firing dropped from 7 of 9 (77.8%) with the full window to 1 of 9 (11.1%) at $m = 10\,\mathrm{s}$ (6 logs lost firing, 0 gained; exact two-sided McNemar $p = 0.03125$). However, this exploratory finding is descriptive rather than confirmatory: across the five diagnostic checks evaluated, an unadjusted $p = 0.031$ does not survive Bonferroni multiplicity correction ($p \approx 0.16$). Furthermore, the other four checks exhibited low baseline sensitivity even on full windows ($\leq 25\%$, with 0/4 for motor and 0/2 for battery on paired logs), meaning they had virtually no baseline firing to lose, while tuning exceeded the declared 10% normal-flight false-alarm ceiling (13%). Thus, while rate-tracking error demonstrated marked leakage, claims that leakage is strictly confined to tuning must be tempered by the limited baseline sensitivity of the remaining checks.

To test diagnostic attribution on independently evaluated incidents, we evaluated the rule across all 43 downloadable logs behind expert forum threads (38 aircraft-fault and 5 pilot-fault). As noted in Section 3, cross-tabulating these 43 logs against archive ratings revealed that all 9 logs carrying an uploader rating were labeled `crash_sw_hw` (and all 9 were diagnosed as aircraft faults by experts, 100% agreement); the 5 expert pilot crashes were submitted unrated, which is consistent with, but not statistical evidence of, reluctance to self-identify pilot fault: all 5 pilot logs were unrated in the index against 28 of 38 aircraft logs unrated (one-sided Fisher exact $p = 0.25$). Evaluated on the 38 aircraft incidents, the fixed rule predicted "aircraft" in 18 (47.4%), "unknown" in 6 (15.8%), and erroneously blamed the "pilot" in 14 (36.8%). On decided aircraft cases ($n = 32$), the rule was wrong on 43.8% (14/32). Detailed inspection of flight mode sequences confirmed the failure mechanism: in all 14 misclassified logs, the vehicle was in a pilot-flown mode at the crash moment (`POSCTL` in 7 logs, `STAB` in 4, `ALTCTL` in 3; including log `e9034973`, which transitioned `POSCTL` $\rightarrow$ `AUTO_4` $\rightarrow$ `POSCTL` as the pilot attempted manual recovery). Because no hardware failsafe fired before impact, the rule defaulted to blaming the pilot.

Conversely, when evaluated on the 5 pilot-fault incidents, the fixed rule predicted "aircraft" in 4 (80.0%) and "pilot" in only 1 (20.0%; pilot recall 1 of 5, exact Clopper–Pearson 95% CI 0.5–71.6%, Wilson 3.6–62.4%), misclassifying 4 of 5 pilot crashes because motor saturation, accelerometer clipping, or failsafe activation occurred as the vehicle struck the ground. Combining both classes across all 37 decided expert logs (32 aircraft, 5 pilot), the rule pro-

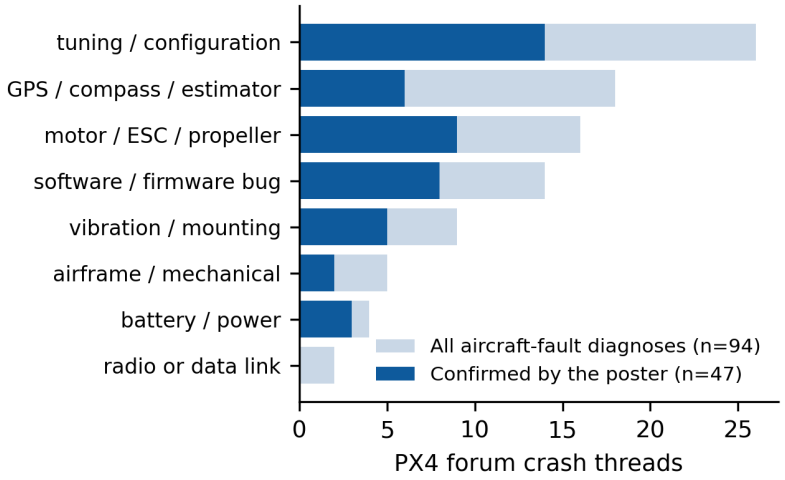


Figure 6: Causes named by experts in PX4 forum threads that attributed a crash to the aircraft.

duced 18 incorrect decisions out of 37 (48.6% error rate), yielding a balanced accuracy of $(56.25\% + 20.0\%)/2 = 38.1\%$ (exact Clopper–Pearson bound 0.19–0.73; Bayesian 95% credible interval 0.25–0.61 under Jeffreys prior), which does not differ significantly from chance (0.50). Similarly, when evaluated out-of-fold on the 30 aircraft logs with complete telemetry features, the logistic regression model predicted "aircraft" in 27 (90.0%) and "pilot" in 3 (10.0%), defaulting strongly toward its aircraft training prior (72.3% average across 5-fold CV training folds; 75.7% on the full 148-crash set, 112/148). We emphasize the caveat that this evaluation measures agreement between a heuristic rule and expert community reading of the same telemetry, rather than ground truth: original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft-fault threads (50.0%) and 5 of 9 pilot-fault threads (55.6%). Nonetheless, this two-class comparison illustrates the challenge of rule-based attribution: the heuristic rule disagreed with expert community consensus on nearly half of decided cases (48.6%), failing to demonstrate agreement above chance.

5.7. Exploratory evaluation of language-model review panels (preliminary)

Because language-model assistants are increasingly proposed for automated flight log analysis, we conducted a preliminary exploratory evaluation on 60 author-generated fault cases using four local models (3–4B parameters)

configured both as single agents and as multi-agent review panels (Table 9). While a fixed deterministic rule achieved 75% accuracy (naming the true cause in 35 of 50 active cases and correctly abstaining on 10 no-cause cases) with 0% error (referring 15 cases to human operators), local language models produced substantial error rates (10–63% wrong), suffered from mode collapse or complete abstention on active causes, and proved highly vulnerable to consensus manipulation: in a worst-case sensitivity sweep, a single compromised reviewer systematically flipped the panel's consensus decision in 45 to 56 of the 60 cases. In contrast, deterministic numeric telemetry extraction into code prevented direct prompt injection by construction across 40 hostile input strings. These preliminary results emphasize that multi-agent consensus voting without cryptographic proofs or fact-citation constraints cannot provide dependable safety assurance.

5.8. Analyses added during revision (exploratory)

To preserve the boundary between pre-declared confirmatory hypotheses and exploratory investigations added during revision [17], Table 10 documents each analysis added during peer review, its motivation, key results, git commit hash, first archive date in Software Heritage, and analysis run date. Confirmatory tests were declared prior to analysis in version control; exploratory analyses added during review are marked with † throughout the text and tables. None of these exploratory additions altered the conclusions of the pre-declared tests; rather, they established empirical bounds on between-airframe confounding, tested alternative architectures, and quantified operational fleet screening performance.

Table 3: Analysis samples and evaluation cohorts.

Sample	Crashes	Comparison	Question
Settings	100 (57 with default records)	477 normal (113 with default records)	Q1
Development crashes	93 analysed	–	Q2 rule
Held-out crashes	112 analysed	143 normal (full logs)	Q2 replication
Leak-curve set	148 at $m = 0\,\mathrm{s}$ (89 at $m = 30\,\mathrm{s}$)	139 normal (124 at $m = 30\,\mathrm{s}$)	Q2 model, Q3a
Constant-subset leak curve	89 crashes ($\geq 40\,\mathrm{s}$ duration)	124 normal ($n = 213$ constant across margins)	Q2 model, Q3a
Within-drone rank test	101 selected (61 and 46 analysed)	475 earlier flights of the same drones	Q3b
Within-drone predictive model	67 multi-flight drones (67 pre-crash)	305 earlier flights of same drones ($n = 372$)	Q3b
Between-drone matched sets	66 matched sets (60 and 47 analysed)	124 matched drones, 372 flights	Q3c
Expert forum cases	148 real crash threads	43 logs downloadable (38 aircraft, 5 pilot)	Q4

Table 4: Features computed on each analysis window.

Feature	Definition
Rate-tracking error	RMS of commanded minus measured body rate, roll and pitch (rad s^{-1}; rotary wing)
Tilt (p95)	95th percentile of tilt angle (deg)
Vibration (p95)	95th percentile of the accelerometer vibration metric, or of the estimator vibration estimate on older firmware
Clipping rate	accelerometer clipping events per second
Motor saturation	share of time with any motor output at 95% or more (rotary wing)
Battery minimum	lowest reported remaining charge (%)
Voltage sag	(start voltage − minimum voltage) / start voltage
Poor GPS	share of time with horizontal accuracy above 5 m or fewer than 6 satellites
Failsafe	1 if a failsafe was active at any time
Manual share	share of time in a pilot-flown mode (manual, altitude, position, acro, stabilised)
Stick deflection (p95)	95th percentile of the larger roll or pitch stick deflection

Table 5: Confirmatory tests, their declared pass marks and outcomes. Holm adjustment over the seven p-valued tests.

Test (declared criterion)	Result	Holm p	Outcome
Protections disabled, crash vs normal	51% vs 46%, $p = 0.63$	1.00	No difference
Attribution rule, dev. (BA ≥ 0.70)	BA 0.57, 35% wrong	–	Failed
Attribution rule, held-out	BA 0.47, 47% wrong	–	Failed
Clipping, aircraft vs pilot ($p < 0.025$)	24% vs 8%, $p = 0.045$	0.27	Not supported
Manual mode, pilot vs aircraft ($p < 0.025$)	18% vs 10%, $p = 0.165$	0.73	Not supported
AUC drop from 0 to 10 s (≥ 0.10)	0.88 to 0.87	–	Not supported
Within-drone vibration ($p < 0.025$)	$\bar{r} = 0.48$, $p = 0.67$	1.00	Not supported
Within-drone tracking ($p < 0.025$)	$\bar{r} = 0.50$, $p = 0.47$	1.00	Not supported
Between-drone vibration ($p < 0.025$)	$\bar{r} = 0.67$, $p = 0.0023$	0.016	Supported on older metric only; replication inconclusive
Between-drone tracking ($p < 0.025$)	$\bar{r} = 0.57$, $p = 0.15$	0.73	Not supported
Expert-fault checks, $m = 10$ s ($\geq 50\%$, $\leq 10\%$)	best: vibration 3/5 vs 11%	–	None met

Table 6: Decomposition of model separation between crash and normal flights: sequential effect of grouping folds by vehicle ID, window alignment, removing tilt/stick features, and within-drone evaluation across the multi-flight cohort.

Step	Evaluation protocol specification	Sample flights	ROC AUC
0	Cross-sectional: stratified 5-fold CV	148 crash, 139 normal	0.879
1	Folds grouped by vehicle ID (Group-CV)	148 crash, 139 normal	0.884
2	Window-aligned: crash window removed ($m = 10$ s)	114 crash, 130 normal	0.880
3	Window-aligned: late window removed ($m = 30$ s)	89 crash, 124 normal	0.863
4	Without tilt and stick features (Group-CV, $m = 0$ s)	148 crash, 139 normal	0.831
5	Within-drone evaluation: Group-CV on same drone's flights	67 crash, 305 earlier	0.580

Table 7: Single-feature AUC for separating crash from normal flights at three cut-offs (no fitting; 0.5 is chance; values below 0.5 mean lower in crash flights).

Feature	$m = 0\,\mathrm{s}$	$m = 10\,\mathrm{s}$	$m = 30\,\mathrm{s}$
Rate-tracking error	0.81	0.72	0.69
Vibration (p95)	0.74	0.72	0.77
Manual share	0.67	0.68	0.69
Motor saturation	0.67	0.66	0.65
Clipping rate	0.59	0.54	0.55
Voltage sag	0.55	0.53	0.48
Poor GPS	0.53	0.55	0.54
Failsafe	0.50	0.47	0.47
Battery minimum	0.42	0.40	0.38
Tilt (p95)	0.41	0.36	0.33
Stick deflection (p95)	0.32	0.33	0.35

Table 8: Fault checks on logs whose expert named the matching fault, with the full window ($m = 0\,\mathrm{s}$) and with the last 10 s before the crash removed ($m = 10\,\mathrm{s}$). Declared criterion: at least 50% of diagnosed logs and at most 10% of normal flights. Paired columns show constant logs surviving across both windows.

	Diagnosed ($m = 0\,\mathrm{s}$)		Diagnosed	Paired Δ	Normal comparison	
Expert-named fault	All logs	Paired	$m = 10\,\mathrm{s}$	(p-value)	$m = 0\,\mathrm{s}$	$m = 10\,\mathrm{s}$
Tuning or configuration	10/12 (83%)	7/9 (78%)	1/9 (11%)	-6 ($p = 0.031$)	18/142 (13%)	7/130 (5%)
Vibration or mounting	3/5 (60%)	3/5 (60%)	3/5 (60%)	0 ($p = 1.00$)	7/48 (15%)	4/38 (11%)
GPS, compass or estimator	1/5 (20%)	1/4 (25%)	1/4 (25%)	0 ($p = 1.00$)	10/143 (7%)	5/130 (4%)
Motor, ESC or propeller	1/9 (11%)	0/4	0/4	0 ($p = 1.00$)	8/143 (6%)	5/130 (4%)
Battery or power	0/4	0/2	0/2	0 ($p = 1.00$)	4/143 (3%)	1/130 (1%)

Table 9: Language-model review panels on 60 generated cases: counts of right (R), wrong (W), and deferred (Def) cases, with coverage (Cov., share of cases decided) and accuracy when answering (Acc.). "Right" includes correctly leaving a no-cause case open. Single-agent gemma3 and phi4 answered only the 10 no-cause cases (0% coverage on active causes). Adversary flips show consensus changes caused by a single compromised reviewer in a worst-case sweep.

	Single agent			Review panel			Compromised
Method	R / W / Def	Cov.	Acc.	R / W / Def	Cov.	Acc.	Flips / 60
Fixed rule (no model)	45 / 0 / 15	75%	100%	–	–	–	–
`gemma3:4b`	10 / 0 / 50	17%†	100%†	3 / 36 / 21	65%	7.7%	49
`phi4-mini`	10 / 0 / 50	17%†	100%†	20 / 19 / 21	65%	51.3%	56
`qwen2.5:3b`	9 / 44 / 7	88%	17.0%	10 / 6 / 44	27%	62.5%	54
`qwen3:4b`	0 / 41 / 19	68%	0.0%	17 / 38 / 5	92%	30.9%	45

†Answered only on the 10 no-cause cases where abstention coincided with the ground truth; 0% coverage on active causes.

Table 10: Exploratory analyses added during peer review, their motivation, key results, git commit hashes, first archived dates in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`), and analysis run dates.

Analysis	Motivation	Key result	Commit	Archived	Run
1,000-permutation null (Task B)	Calibrate detection threshold	Null mean 0.497; observed AUC upper bounds 0.510–0.619	`bad06ff`	2026-10-25	2026-10-25
Depth-1 decision stumps (Task B)	Test single-feature non-linear splits	Stumps AUC 0.430 $\rightarrow$ 0.413 at $m = 10\,\mathrm{s}$ (bounds ≤ 0.559)	`bad06ff`	2026-10-25	2026-10-25
Constant-subset leak curve	Exclude sample attrition ($n = 213$)	AUC declines smoothly 0.895 $\rightarrow$ 0.868 ($m = 10\,\mathrm{s}$)	`bbdd042`	2026-10-25	2026-10-25
Paired diagnostic leakage test	Test leakage on constant logs ($n = 9$)	Tuning check 7/9 $\rightarrow$ 1/9, exact McNemar $p = 0.03125$	`bbdd042`	2026-10-25	2026-10-25
Same-sample decomposition	Account for airframe & duration confounds	Group-CV AUC 0.884; within-drone Group-CV 0.580 on same flights	`eb463b5`	2026-10-25	2026-10-25
Poisson-binomial set null	Account for mixed set sizes ($n = 60$)	Exact null rate 40.28%; observed 55.0% ($p = 0.0137$)	`bad06ff`	2026-10-25	2026-10-25
Screening PPV and CIs	Quantify operational fleet value	Fleet PPV 2.26–2.47% at 10% flag rate (3.3$\times$ lift)	`bad06ff`	2026-10-25	2026-10-25
Sensitivity by vibration source	Compare older vs. newer metrics	Older: 32/48 (66.7%); newer: 6/12 (50.0%); pooled newer $\bar{r} = 0.58$ ($p = 0.25$)	`624dd69`	2026-10-25	2026-10-25
Independent 43-log validation	Evaluate against independent causes	Balanced accuracy 38.1% (exact CP 0.19–0.73); pilot recall 1/5	`3a9aa89`	2026-10-25	2026-10-25

6. Design implications for fleet assurance

To translate these empirical findings into operational practice, we developed AeroFleet 2.0 (`https://github.com/AdityaPathare46/aerofleet`), an open-source, ground-side assurance tool for PX4 and ArduPilot fleets. The tool reads existing flight logs and configuration files without executing unverified autonomous decisions or contacting the aircraft in flight. Table 11 maps each empirical finding from Sections 3–5 directly to an assurance design implication.

In accordance with our negative attribution results (Q1–Q2), the tool presents a structured, chronological event review reporting evidence quality (dilution of precision, sensor health, and mode transitions) while strictly abstaining from issuing automated pilot-versus-aircraft liability verdicts. Responding to the vibration finding (Q3), the tool ranks airframes longitudinally against comparable peers within the same fleet to prioritize the top decile (10%) for physical maintenance inspection (operational PPV 2.26–2.47%, lift 3.3×, 40–44 inspections per crash), rather than issuing per-flight crash alarms. Finally, to ensure evidentiary integrity over the lifecycle of each airframe, all reviews, approvals, and configuration checks are appended to a tamper-evident, Ed25519-signed record referencing SHA-256 digests of the original telemetry files.

7. Discussion

Attribution. A declared rule, its held-out replication and cross-validated models all found no usable separation between pilot- and aircraft-caused crashes with these labels and features. Because pilot labels are sparse ($n = 36$ in the modeling sample), AUCs below roughly 0.65 cannot be statistically distinguished from chance at 80% power (permutation 97.5th percentile 0.660); observed cross-validated models yield empirical 95% bootstrap upper bounds between 0.510 and 0.619. An operational attribution verdict would require an AUC of approximately 0.8 or higher, which these models do not approach. Furthermore, independent evaluation against 43 expert forum cases revealed poor agreement on decided cases (balanced accuracy 0.381, exact 95% Clopper–Pearson bound 0.19–0.73, Bayesian 95% CrI 0.25–0.61; pilot recall 1 of 5, exact Clopper–Pearson 95% CI 0.5–71.6%). While commercial and legal disputes often seek a binary verdict, aviation safety science demonstrates that accidents are multi-causal systemic events involving organizational, environmental and human factors [7]. Uploader labels are noisy,

Table 11: From empirical findings to design implications in fleet assurance.

Finding	Empirical result (Section)	Design implication for fleet assurance
Attribution failure	Logs do not separate pilot from aircraft causes above chance (balanced accuracy 0.47 on held-out test, 0.381 on expert set; §**??**, §5.6)	Present time-ordered event chronology and data quality metrics; abstain from automated fault verdicts.
Diagnostic leakage	Apparent diagnostic accuracy drops once crash window is removed (tuning check sensitivity falls 77.8% $\rightarrow$ 11.1%; §5.3, §5.6)	Remove crash impact window ($m \geq 10\,\mathrm{s}$) and compare against aircraft operating baseline before diagnosing faults.
Between-drone vibration	Crashed drones show higher baseline vibration on older estimator metrics (mean rank 0.67, Holm $p = 0.016$; §**??**)	Rank drones longitudinally against fleet peers; prioritize top decile (10%) for mechanical maintenance inspection.
Confounding shrinkage	Cross-sectional separation collapses from 0.88 to 0.58 under within-drone evaluation on same flights (§**??**)	Evaluate airframe health against its own flight history, not cross-sectional fleet averages.
Audit integrity	Multi-agent consensus voting is vulnerable to rogue reviewer manipulation (§5.7)	Rely on deterministic numeric telemetry extraction and append reviews to a tamper-evident, signed log.

and the divergence between uploader pilot blame (32%) and expert forum diagnoses (6%) may reflect several untested mechanisms, including differing definitions of pilot responsibility among autopilot developers or differing thresholds for public fault attribution. A log records the combined physical outcome of pilot inputs, environmental disturbances and vehicle response; disentangling responsibility from telemetry alone is not supported by the data. Tools that issue automatic responsibility verdicts should be evaluated against independent investigations before adoption.

Leakage and confounding. While cross-sectional models showed high apparent separation (AUC 0.88), this was likely influenced by airframe confounds and duration differences (duration alone yields AUC 0.662); within-drone evaluation demonstrated substantial attenuation (AUC 0.580, 95% CI 0.528–0.633), confirming that longitudinal baseline comparison is essential. Diagnostic checks were also *consistent with* crash-window leakage: specifically, tuning fault detection fell from 77.8% to 11.1% sensitivity ($p = 0.031$ on paired logs) when the final 10 s were removed, though this finding is exploratory, does not survive Bonferroni correction over five checks ($p \approx 0.16$), and the low baseline sensitivity of the remaining checks ($\leq 25\%$) leaves open whether other physical failure modes exhibit the same pattern. Regardless of the precise mechanism, diagnostic evaluations must remove the crash window and compare against an aircraft's own operating baseline before claiming pre-crash anticipation.

Drone-level risk. The positive result concerns airframes, not flights. Drones that later crashed vibrated more than comparable drones on ordinary flights, consistent with a maintenance view in which mechanical degradation precedes catastrophic failure [24]. However, two important qualifications apply. First, removing crash-day flights reduced usable sets from 60 to 25, indicating that many flights occurred during the same session; the "maintenance degradation" hypothesis therefore competes with an "experimental airframe" hypothesis, where poorly tuned or prototype drones vibrate heavily and crash during initial testing. Second, the effect was supported on the older estimator metric only; a declared replication on the newer accelerometer metric yielded an independent estimate of $\bar{r} = 0.50$ ($p = 0.5760$, 10 new sets) and pooled $\bar{r} = 0.58$ ($p = 0.251$, 19 sets), which is inconclusive and severely underpowered ($N = 247$ sets needed at true $\bar{r} = 0.58$ due to potential winner's curse). Finally, across the 0.689% screenable multi-flight base rate and

0.741% fleet vehicle base rate, stratum-matched fleet screening prioritizes the top decile with an operational PPV of 2.26–2.47% (lift 3.3×, 40.5–44.2 inspections per detected crash), and the top quartile with PPV 1.37–1.48% (lift 2.0×, 67.5–73.1 inspections per crash; 0.96–1.04% under empirical matched-set screening). The metric provides a maintenance prioritization signal, not an operational warning.

From findings to assurance design. Section 6 translates these empirical findings into design implications for fleet assurance: structured evidence review without automated verdicts, longitudinal airframe ranking, and tamper-evident telemetry records.

Language models. On this task small local models added errors and voting vulnerabilities rather than accuracy. While deterministic numeric parsing prevented direct prompt injection by construction, multi-agent voting consensus was easily manipulated by a single compromised reviewer. Larger models and blind evaluation cases remain to be tested.

Relation to prior datasets. Benchmarks built from injected faults [10, 11] answer whether a known fault can be detected; anomaly datasets from real logs [12] answer whether unusual segments can be found. Our results concern a different question, attribution of a real crash, and suggest that performance on the first two does not transfer to it: signals that mark a fault also mark the crash it causes and the pilot's reaction to it. Datasets that pair real crashes with independently verified causes would be needed to make progress on attribution, and our expert-diagnosis labels are a small first step.

Records and regulation. Record-keeping rules ask operators to keep logs, maintenance records and occurrence reports [1, 2]. On our evidence, such records are most useful when they are kept per aircraft over time, so that an airframe's condition can be compared with its own history and with comparable airframes, rather than examined only after an accident.

Practical guidance. For operators and investigators we suggest: (i) treat automatic pilot-or-aircraft verdicts from logs with caution; (ii) use logs to assemble a time-ordered record of events for a human investigator; (iii) track vibration per drone across flights and inspect drones that are rough relative to comparable airframes; (iv) when evaluating any log-based diagnostic, remove the crash window and compare against the same drone's history.

Future work. Two studies have been declared in the repository before any data were collected: a prospective test in partner fleets, in which the monthly vibration ranking of AeroFleet 2.0 is written to the signed record before outcomes occur and crashes, unplanned repairs and defects found at inspection are compared between flagged and unflagged drones over six months; and a within-subjects user study comparing the three-dimensional views with a two-dimensional map on tasks with known answers. Stronger replication of the between-drone vibration effect on modern autopilots, evaluating operational safety impact across active fleets, testing sequence models or frequency-domain IMU features for transient precursors that whole-window summaries miss, and evaluating a larger set of crashes with causes verified by independent investigators would further strengthen the evidence.

8. Threats to validity

Archive self-selection. The public Flight Review archive reflects voluntary uploads from hobbyists, academic researchers, and commercial operators, introducing self-selection towards unusual flights, developer tuning sessions, and unexplained incidents. Crashed airframes and normal comparison flights differ substantially in flight duration (median 54 s vs. 388 s), firmware era, and operational envelope, which motivated our within-drone and matched-pair designs.

Labels. The uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments, and normal flights are self-rated. Across 148 expert forum crash threads, download attempts yielded 43 downloadable logs (38 aircraft-fault, 5 pilot-fault), enabling a two-class empirical validation that revealed an overall error rate of 48.6% on decided cases (balanced accuracy 0.381, exact 95% Clopper–Pearson bound 0.19–0.73, Bayesian 95% CrI 0.25–0.61; pilot recall 1 of 5, exact Clopper–Pearson 95% CI 0.5–71.6%; uploader `crash_pilot` rating 0 of 43). However, forum diagnoses represent informed community opinion rather than official accident investigation reports; original posters confirmed the diagnosis in only 47 of 94 aircraft threads (50.0%) and 5 of 9 pilot threads (55.6%). Controls without a crash rating may have crashed unnoticed, biasing the between-drone test conservatively. Forum diagnoses achieved good inter-rater agreement ($kappa \geq 0.73$), but were restricted to public discussions.

Confounding and design. Matching on vehicle type, controller hardware and PX4 version leaves open differences in propellers, payload, flight style or operator identity, leaving potential unmodeled clustering. Results may not transfer to commercial ArduPilot fleets or to proprietary autopilots, even though the tool is designed to support both ArduPilot and PX4 logs.

Measurement. The crash moment is estimated; the within-drone result did not depend on the tilt threshold (mean scaled rank for S7 rate-tracking error was 0.46 at 45°, 0.48 at 60°, and 0.50 at 75°, all $p \geq 0.53$). The between-drone effect was shown mainly with the older vibration estimate and was not confirmed on the newer metric in a declared replication with 19 usable sets; this main positive result needs stronger replication on newer firmware.

Protocols and registration. Protocols were declared and committed to version control in advance of analysis, and archived snapshots were deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`); however, commit timestamps in self-hosted git remain author-controlled, distinguishing declared protocols from formal third-party timestamped registries (such as the Open Science Framework). One amendment (lifting the within-drone firmware filter) was data-dependent after observing sample starvation ($n = 12$ under the initial ≥ 1.10 filter).

Features. Summary features over the whole window (percentiles, shares of time, RMS) were used; sequence models or frequency-domain IMU features might capture short precursors that summaries miss. This was not tested.

Attrition. Across 148 real crash threads, download attempts yielded 43 downloadable logs; exactly 35 of the 71 candidate logs behind expert aircraft diagnoses with a checkable cause could still be downloaded, so several diagnostic comparisons in Table 8 have very small sample sizes ($n = 2$ to 9); its results are indicative rather than definitive.

Power and sizing. Some comparisons are small (14 pilot-caused matched sets, 12 sets with the newer vibration metric, five vibration-diagnosed logs). Null results there are not evidence of no effect. For the newer accelerometer vibration metric, the independent sample of 10 new cases yielded $\bar{r} = 0.50$

($p = 0.5760$), and the pooled 19 sets yielded $\bar{r} = 0.58$ ($p = 0.251$). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if $\bar{r} = 0.58$). Detecting $\bar{r} = 0.67$ with 80% power requires 55 sets assuming the discovery effect was uninflated, but if winner's curse inflated the initial estimate and the true effect is $\bar{r} = 0.58$, 247 matched sets are required. Because 247 sets exceed the capacity of the public archive, definitive replication requires multi-fleet partner deployments.

Language models. The AI comparison is preliminary: the experiments use 60 author-generated cases and relatively small local models (3–4B parameters). The fixed rule's performance may be optimistic on cases created by the same authors who designed the rule (0 wrong answers is close to a training score); a blind set of cases written by someone who has not seen the rule is in preparation. Larger models or alternative prompt architectures were not tested.

The tool. AeroFleet 2.0's checks are primarily engineering tests on archive data and synthetic fault injections, not evidence of improved operational safety. Plans in the injection test were built from the flights themselves; the description search was tested on forum titles, which are shorter and more varied than an operator's report, and its symptom lists were written by the authors; prospective fleet evaluation of the vibration ranking and user studies of the three-dimensional views remain future work (Sections 6 and 7).

9. Conclusion

Across 590 crash-rated logs from an archive index of 479,547 flights, flight logs did not distinguish pilot-caused from aircraft-caused crashes above chance: cross-validated AUCs (0.41–0.50, upper bounds ≤ 0.619) fall far short of the ≈ 0.80 needed for operational verdicts, with only AUCs above ≈ 0.65 detectable at 80% power under 36 pilot crashes. Two-class expert validation yielded poor agreement (balanced accuracy 0.381, exact 95% Clopper–Pearson bound 0.19–0.73; pilot recall 1 of 5). Configuration protections were disabled at similar rates, and the flight that crashed showed no detectable difference in vibration or rate-tracking error relative to the same drone's prior flights (95% CI upper bounds 0.58 and 0.63). Diagnostic checks also suffered from crash-impact data leakage: tuning fault detection fell from 77.8% to 11.1% sensitivity ($p = 0.031$ on paired logs) once the final 10 s were removed. What the logs did reveal, on older estimator metrics, was that drones

that later crashed exhibited higher baseline vibration on ordinary flights than matched peers, though replication on newer accelerometer metrics was inconclusive ($p = 0.25$). Screening against stratum-matched fleet controls achieves an operational positive predictive value of 2.26–2.47% (lift 3.3×, 40.5–44.2 inspections per crash) at the top decile and 1.37–1.48% (lift 2.0×) at a 25% flag rate across screenable (0.69%) and fleet-wide (0.74%) vehicle base rates (0.96–1.04% under empirical matched-set screening). Flight log analytics for drone fleets should prioritize longitudinal airframe screening and structured evidence review, not automated verdicts. AeroFleet 2.0 implements this approach as an open-source tool, presenting verifiable evidence while leaving diagnostic decisions to human operators.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The authors develop the open-source AeroFleet 2.0 tool described in Section 6; it is not commercial.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write analysis code, draft and revise text, and assist the recording of forum diagnoses. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

Selection scripts, analysis code, the declared hypotheses with their commit history, derived features, expert labels (without forum text), and the source code of the AeroFleet 2.0 tool are available at `https://github.com/AdityaPathare46/aerofleet`; a DOI-archived release will be provided on acceptance. Raw logs are public on PX4 Flight Review and are referenced by identifier.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] European Commission, Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft, Official Journal of the European Union, L 152, 11 June 2019 (2019).

[2] European Union Aviation Safety Agency, Easy Access Rules for Unmanned Aircraft Systems (Regulations (EU) 2019/947 and 2019/945), EASA, Cologne, revision of June 2026 (2026).

[3] Ministry of Civil Aviation, Government of India, The Drone Rules, 2021, G.S.R. 589(E), Gazette of India, 25 August 2021 (2021).

[4] Federal Aviation Administration, 14 CFR Part 107: Small Unmanned Aircraft Systems, Electronic Code of Federal Regulations, as of 21 October 2026 (2026).

[5] V. S. A. Duvvuru, B. Zhang, M. Vierhauser, A. Agrawal, LLM-agents driven automated simulation testing and analysis of small uncrewed aerial systems, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 385–397. doi:10.1109/ICSE55347.2025.00223.

[6] S. Silalahi, T. Ahmad, H. Studiawan, F. Breitinger, DroPTC: Sentence-level drone flight log forensics using contrastive learning and explainable AI, Forensic Science International: Digital Investigation 56 (2026) 302051. doi:10.1016/j.fsidi.2026.302051.

[7] D. A. Wiegmann, S. A. Shappell, A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System, Ashgate, Aldershot, 2003.

[8] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions

on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[9] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[10] A. Keipour, M. Mousaei, S. Scherer, ALFA: A dataset for UAV fault and anomaly detection, The International Journal of Robotics Research 40 (2–3) (2021) 515–520. doi:10.1177/0278364920966642.

[11] X. Le, B. Jin, G. Cui, X. Dai, Q. Quan, RflyMAD: A dataset for multicopter fault detection and health assessment, The International Journal of Robotics Research 44 (7) (2025) 1081–1092. doi:10.1177/02783649241305153.

[12] A. Kabaoglu, S. Sariel, UAV-SEAD: State estimation anomaly dataset for UAVs, arXiv preprint https://arxiv.org/abs/2602.13900 (2026).

[13] G. Wild, J. Murray, G. Baxter, Exploring civil drone accidents and incidents to help prevent potential air disasters, Aerospace 3 (3) (2016) 22. doi:10.3390/aerospace3030022.

[14] Air Accidents Investigation Branch, Annual Safety Review 2025, Tech. rep., AAIB, Department for Transport, Farnborough, UK, published 4 June 2026 (2026).

[15] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[16] PX4 Development Team, Flight Review log archive and pyulog ULog parser, https://review.px4.io; pyulog v1.2.4, https://github.com/PX4/pyulog; BSD-3-Clause (2026).

[17] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606. doi:10.1073/pnas.1708274114.

[18] J. Wei, H. Li, Y. Li, Y. Zhao, D. Yao, B. Lu, X. Ye, S.-K. Im, Y. Wang, X. Yang, AeroTSBoost: Temporal-statistical boosting for real-world UAV telemetry anomaly mining, arXiv preprint https://arxiv.org/abs/2605.25639 (2026).

[19] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, A. Rasheed, A UAV testbed for diagnosing hardware vulnerabilities: Quantifying sim-to-real discrepancies in PX4 flight logs, Sensors 26 (10) (2026) 3188. doi:10.3390/s26103188.

[20] R. Puchalski, W. Giernacki, UAV fault detection methods, state-of-the-art, Drones 6 (11) (2022) 330. doi:10.3390/drones6110330.

[21] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (1) (2018) 9:1–9:24. doi:10.1145/3146389.

[22] E. Zibaei, R. Borth, Building causal models for finding actual causes of unmanned aerial vehicle failures, Frontiers in Robotics and AI 11 (2024) 1123762. doi:10.3389/frobt.2024.1123762.

[23] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, An exploratory study of autopilot software bugs in unmanned aerial vehicles, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 20–31. doi:10.1145/3468264.3468559.

[24] E. Petritoli, F. Leccese, L. Ciani, Reliability and maintenance analysis of unmanned aerial vehicles, Sensors 18 (9) (2018) 3171. doi:10.3390/s18093171.

[25] A. P. Tvaryanas, W. T. Thompson, S. H. Constable, Human factors in remotely piloted aircraft operations: HFACS analysis of 221 mishaps over 10 years, Aviation, Space, and Environmental Medicine 77 (7) (2006) 724–732.

[26] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 11733–11763.
URL https://proceedings.mlr.press/v235/du24e.html

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[28] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, IEEE Transactions on Neural Networks and Learning Systems 25 (5) (2014) 845–869. doi:10.1109/TNNLS.2013.2292894.

[29] E. A. Stuart, Matching methods for causal inference: A review and a look forward, Statistical Science 25 (1) (2010) 1–21. doi:10.1214/09-STS313.

[30] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59. doi:10.3233/AIC-1994-7104.

[31] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389. doi:10.1561/1500000019.

[32] R. Stoakley, M. J. Conway, R. Pausch, Virtual reality on a WIM: Interactive worlds in miniature, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '95), 1995, pp. 265–272. doi:10.1145/223904.223938.

[33] J. A. Hanley, B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology 143 (1) (1982) 29–36. doi:10.1148/radiology.143.1.7063747.

[34] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[35] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[36] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL https://www.jstor.org/stable/4615733